A3 pratique — Un microscope bruyant

Classification des cellules sanguines par un CNN et restauration RGB par un GRU

Noms : … **Date :** …

Durée : 3 heures. Complétez le code TODO et les réponses écrites en anglais. Utilisez BloodMNIST : images microscopiques couleur de huit catégories de cellules. Le CNN classe les cellules ; un GRU unidirectionnel distinct restaure les images bruitées artificiellement. Consultez [lien manquant] `ASSIGNMENT_EN_CNN_Improved.md`. Conservez les partitions fournies et utilisez la validation pour la sélection du modèle. Programme : données 15 min · CNN 65 min · bruit 15 min · GRU 50 min · évaluation 25 min · conclusion 10 min. Veuillez noter qu'il s'agit d'une expérience pédagogique de traitement d'images.

0. Configuration — fournie

Installez les prérequis avant le cours. Exécutez le notebook depuis ce dossier. Le sous-ensemble RGB fourni permet une exécution hors ligne.

Dans [1…

```
from pathlib import Path import copy import time import random import numpy as n
```

Dans [2…

```
# Le chargement du jeu de données utilise le script de préparation fourni. SEED
( ' cuda ' if torch.cuda.is_available ( ) else ' cpu ' ) # utiliser mps pour mac
```

```
Périphérique : processeur
Modèles améliorés BloodMNIST : filtres CNN 32/64 avec BatchNorm et dropout ; tail
le cachée GRU unidirectionnelle 128.
```

Dans [3…

```
cache = Path('data/blood_mnist_subset.npz') # Spécifiez le chemin vers vo
si cache n'existe pas : from prepare_data import main as prepare_dataset prepa
```

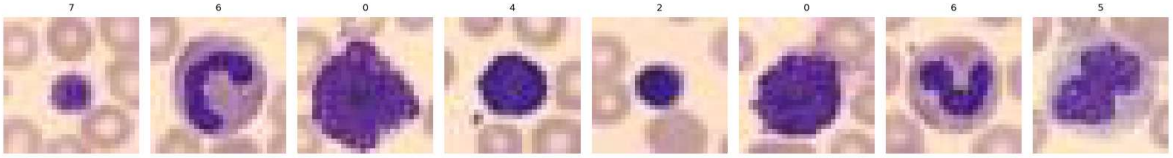
```
    et X.max() <= 1
cnn_train_ids = np.arange(3200)
cnn_val_ids = np.arange(3200, 3840)
cnn_test_ids = np.arange(3840, 4640)
cnn_train_loader = DataLoader(TensorDataset(X[cnn_train_ids], y[cnn_
                              batch_size=BATCH_SIZE, shuffle=True, num_w
cnn_val_loader = DataLoader(TensorDataset(X[cnn_val_ids], y[cnn_val_
cnn_test_loader = DataLoader(TensorDataset(X[cnn_test_ids], y[cnn_te
# Les deux tâches utilisent les mêmes images mises de côté ; les étiquettes de c
den_train_ids, den_val_ids, den_test_ids = cnn_train_ids, cnn_val_ids,
clean_train, clean_val, clean_test = X[den_train_ids], X[den_val_ids
den_train_loader = DataLoader(TensorDataset(clean_train), batch_size=B
print('Images:', len(clean_train), len(clean_val), len(clean_test
print('Identifiants et noms des classes:', dict(énumérer(CLASS_NAMES)))
```

```
Images : 3200 640 800
Identifiants et noms des classes : {0 : 'Basophile', 1 : 'Éosinophile', 2 : 'Éryt
hroblaste', 3 : 'Granulocytes immatures', 4 : 'Lymphocyte', 5 : 'Monocyte', 6 :
'Neutrophile', 7 : 'Plaquette'}
```

Dans [4…

```
def show_images(images, titles=None, n=8): n = min(n, len(images))
```

```
afficher_images(X[cnn_train_ids[:8]], y[cnn_train_ids[:8]].toli
print('Tenseur complet :', tuple(X.shape)) print('Mini-lot :', tuple
```




```
Tenseur complet : (4640, 3, 28, 28)
Mini-lot : (128, 3, 28, 28)
```

Q1 — Données et généralisation

1. Expliquez chaque dimension du mini-lot.

2. Pourquoi les deux tâches doivent-elles exclure les mêmes images de l'entraînement ?
3. Que représentent les huit identifiants de classe ? Donnez trois exemples.

Vos réponses : ...

Q1.

```
1 The mini-batch tensor has shape (128, 3, 28, 28),
corresponding to (B, C, H, W):

B = 128: the batch dimension. It is not 128 separate batches,
but a single batch containing 128 images processed together in
one forward/backward pass.
C = 3: the number of colour channels (Red, Green, Blue). Each
channel stores the intensity of that colour component at every
pixel.
H = 28, W = 28: the spatial dimensions — height and width of
each image in pixels.

2 This ensures that no image used to evaluate either model was
seen during training by that model, avoiding data leakage. It
also makes the two tasks directly comparable: since both are
evaluated on exactly the same held-out images, differences in
performance reflect the models and tasks themselves, not
differences in the data each one happened to see

3 The eight class IDs are numerical labels (0 to 7) representing
the eight blood cell categories defined in CLASS_NAMES. For
example:

ID 0 → Basophil, a type of white blood cell involved in allergic
and inflammatory responses.
ID 4 → Lymphocyte, a white blood cell central to the immune
response.
ID 7 → Platelet, a cell fragment involved in blood clotting.
```

1. CNN — formes et comptage manuel des paramètres (45 min)

Utilisez deux blocs de convolution (noyau 3, pas 1, remplissage 1), chacun suivi d'une normalisation par lots 2D, d'une fonction d'activation ReLU et d'un MaxPool2D(2). Ajoutez une couche linéaire cachée de 128 unités, une fonction d'activation ReLU, un taux d'abandon de 0,25 et huit logits de sortie. Toutes les couches de convolution et linéaires incluent des biais.

Tâche de recherche — deux nouvelles couches (10 minutes dans la section CNN)

Avant d'utiliser ces couches fournies, consultez la documentation officielle de PyTorch pour BatchNorm2d et Dropout .

1. En vos propres termes, quel est le rôle de chaque couche ? En quoi pourrait-elle être utile à un classificateur ?

BatchNorm2d normalise les activations de chaque canal (moyenne nulle, variance unitaire). Cela stabilise et accélère l'entraînement, réduit le décalage de covariable interne et permet des taux d'apprentissage plus élevés.

Le dropout désactive aléatoirement une partie des activations pendant l'entraînement, empêchant ainsi le réseau de dépendre excessivement de certains neurones. Cela réduit le surapprentissage en limitant la coadaptation entre les unités.

2. Comment chaque couche se comporte-t-elle dans les modes `model.train()` et `model.eval()` ?

BatchNorm2d : en mode entraînement, il calcule la moyenne et la variance à partir du lot courant et met à jour ses statistiques cumulées. En mode évaluation, il utilise la moyenne et la variance cumulées, en ignorant le lot courant.

Dropout : actif uniquement en mode entraînement, il met les unités à zéro de manière aléatoire. En mode évaluation, il est totalement désactivé ; toutes les unités sont utilisées normalement.

3. Quelles `p=0.25` conséquences cela a-t-il pour les personnes qui abandonnent leurs études ? Qu'advient-il des activations conservées ?

Chaque unité a 25 % de chances d'être désactivée à chaque itération lors de l'entraînement. Les unités actives restantes sont multipliées par 1/(1-0,25) = 1,33 afin de maintenir la somme d'activation attendue approximativement constante ; aucun ajustement n'est donc nécessaire lors de l'évaluation.

4. Quelles valeurs sont apprises par BatchNorm ? Quelles sont ses statistiques d'exécution ? Dropout possède-t-il des paramètres entraînables ?

BatchNorm2d apprend deux paramètres entraînables par canal (γ et β), mis à jour par rétropropagation. Sa moyenne et sa variance mobiles sont des tampons, et non des paramètres ; elles sont mises à jour par une moyenne mobile, et non par descente de gradient. Le dropout, quant à lui, ne possède aucun paramètre entraînable : il s'agit d'une opération stochastique fixe.

5. Expliquez pourquoi l'évaluation doit appeler `model.eval()` même à l'intérieur `torch.no_grad()` .

La fonction `torch.no_grad()` désactive uniquement le suivi des gradients pour économiser de la mémoire et des ressources de calcul ; elle ne modifie pas le comportement des couches. C'est la fonction `model.eval()` qui active le mode d'inférence pour BatchNorm2d et Dropout. Sans elle, même sans calcul de gradients, les résultats resteraient incohérents en raison de la normalisation dépendante des lots et du dropout aléatoire actif.

Mes liens sources : https://moodle-ovh.isep.fr/moodle/pluginfile.php/132422/mod_resource/content/1/Chapter1.pdf , https://docs.pytorch.org/docs/stable/generated/torch.nn.BatchNorm2d.html , https://docs.pytorch.org/docs/stable/generated/torch.nn.Dropout.html

Veuillez compléter ce tableau avant d'utiliser le compteur automatique :

Couche	Forme de sortie	Paramètres paramétrables manuellement
Entrée, mise à l'échelle fixe à [-1,1]	(B,3,28,28)	0
Conv2d 3 → 32	(B, 32, 28, 28)	896
BatchNorm2d(32), ReLU	(B, 32, 28, 28)	64
MaxPool2d	(B, 32, 14, 14)	0

| Conv2d 32 → 64 | (B,64,14,14) | 18496 | | BatchNorm2d(64), ReLU | (B,64,14,14) | 128 | | MaxPool2d | (B,64,7,7) | 0 | | Flatten | (B,3136) | 0 | | Linear → 128, ReLU | (B,128) | 401,536 | | Dropout(0.25) | (B,128) | 0 | | Linear → 8 logits | (B,8) | 1,032 | | **Total** | — | 422,152 |

Convolution : Cout(Cin k^2+1). Linéaire : dout(din+1). Normalisation par lots : 2C (mise à l'échelle et décalage). Expliquez pourquoi le dropout, la fonction ReLU et le pooling n'ajoutent aucun paramètre appris.

ReLU, MaxPool2d et Dropout sont des opérations fixes, sans paramètre, définies par une règle mathématique plutôt que par des poids appris. ReLU remplace simplement les valeurs négatives par zéro (max(0, x)). MaxPool2d sélectionne la valeur maximale dans chaque fenêtre locale. Dropout annule aléatoirement les activations selon une probabilité fixe p. Aucune de ces opérations n'implique une matrice de poids ou un vecteur de biais pouvant être mis à jour par rétropropagation ; elles filtrent, sélectionnent ou remodèlent uniquement les valeurs déjà produites par la couche précédente (paramétrée).

Dans [5…

```
class SmallCNN ( nn.Module ) : def __init __ ( self ) : super ( ) .__ init __ (
```

```
    def forward ( self , x ): x = ( x - 0.5 ) / 0.5 # Rééchelonnement fixe de l'
```

Dans [6…
```
cnn  =  SmallCNN ( ) . to ( device )
assert  cnn ( torch.zeros ( 3 , 3 , 28 , 28 , device = device )  ) . shape == (



# À exécuter uniquement après avoir écrit le calcul manuel ci-dessus. def count_
```

```
Nombre de paramètres du CNN : 422 152
```

2. Entraînement et évaluation du CNN (20 min)

Complétez la fonction d'assistance à la mise à jour du gradient. Elle sera également réutilisée pour le débruitage. La boucle fournie évalue la perte de validation et conserve le meilleur point de contrôle.

Q2

Pourquoi `CrossEntropyLoss` utiliser les logits plutôt que les probabilités softmax ? Quelle est la précision d'une distribution uniforme aléatoire pour 8 classes ? Que `train()` font `eval()` et `no_grad()` ?

La fonction CrossEntropyLoss de PyTorch combine en interne la fonction LogSoftmax et la perte de vraisemblance logarithmique négative en une seule opération numériquement stable. Si les probabilités softmax étaient fournies à la place des logits bruts, la fonction appliquerait la fonction softmax deux fois, ce qui fausserait à la fois la valeur de la perte et les gradients. Le calcul direct de la fonction log-softmax à partir des logits évite également les problèmes numériques (dépassement de capacité) qui peuvent survenir si le calcul de la fonction softmax et de son logarithme était effectué en deux étapes distinctes.

Avec 8 classes équilibrées, une répartition aléatoire uniforme donne une précision attendue de 1/8 = 12,5 %. Tout modèle entraîné devrait clairement dépasser cette valeur de référence.

La méthode `model.train()` active le mode d'entraînement du modèle : BatchNorm2d utilise les statistiques du lot actuel et met à jour ses moyennes mobiles, tandis que Dropout met aléatoirement à zéro certaines unités. La méthode `model.eval()` active le mode d'évaluation du modèle : BatchNorm2d utilise alors ses statistiques mobiles cumulées au lieu des statistiques du lot, et Dropout est désactivé. La méthode `torch.no_grad()` est un gestionnaire de contexte distinct qui désactive le calcul et le suivi du gradient, réduisant ainsi la consommation de mémoire et de ressources de calcul pendant l'inférence. Elle n'affecte pas le comportement de BatchNorm2d ni de Dropout ; elle est généralement utilisée conjointement avec `model.eval()`, et non en remplacement de ce dernier.

Dans [7…
```
def update_batch ( model , inputs , targets , optimizer , criterion ): optimizer
```

```
cnn_criterion  =  nn.CrossEntropyLoss ( ) # TODO T6 cnn_optimizer = cnn_optimize
```

Dans [8…

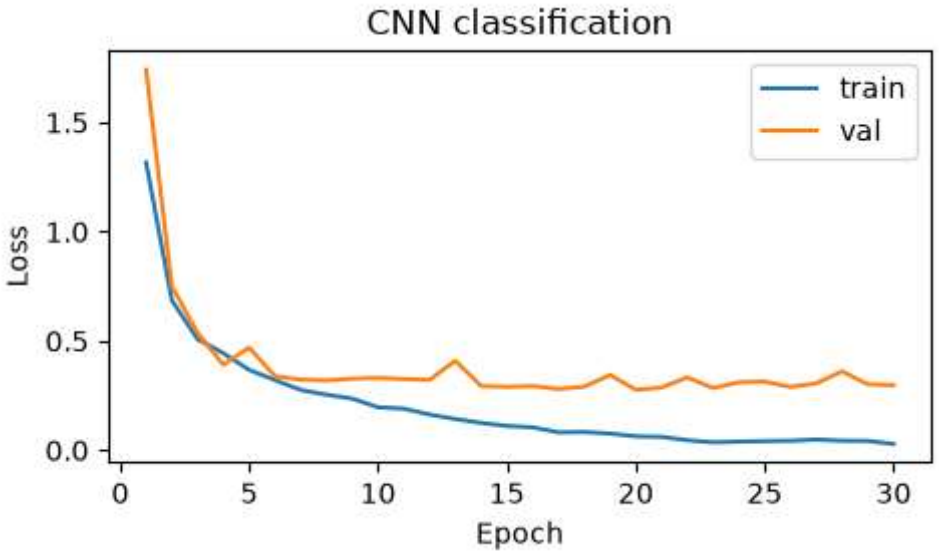
```
def fit_classifier ( model , train_loader , val_loader , optimizer , criterion ,
```

```
            best_loss  =  val_loss ;  best_weights  =  copy . deepcopy ( model .
        print ( f 'Époque { epoch + 1 } : train= { total / count : .4f } , val=
    model . load_state_dict ( best_weights )
    return  history

def plot_history ( history , title ) : plt.figure ( figsize = ( 5 , 3 ) ) for ke
```

```
started  =  time.perf_counter () cnn_history = fit_classifier ( cnn , cnn_train_
```

```
Époque 1 : train=1,3171, val=1,7419
Époque 2 : train=0,6857, val=0,7506
Époque 3 : train=0,5081, val=0,5363
Époque 4 : train=0,4441, val=0,3907
Époque 5 : train=0,3674, val=0,4699
Époque 6 : train=0,3211, val=0,3389
Époque 7 : train=0,2759, val=0,3229
Époque 8 : train=0,2539, val=0,3198
Époque 9 : train=0,2362, val=0,3276
Époque 10 : train=0,1967, val=0,3309
Époque 11 : train=0,1902, val=0,3260
Époque 12 : train=0,1632, val=0,3216
Époque 13 : train=0,1430, val=0,4095
Époque 14 : train=0,1249, val=0,2940
Époque 15 : train=0,1113, val=0,2901
Époque 16 : train=0,1048, val=0,2939
Époque 17 : train=0,0818, val=0,2804
Époque 18 : train=0,0835, val=0,2906
Époque 19 : train=0,0763, val=0,3448
Époque 20 : train=0,0636, val=0,2767
Époque 21 : train=0,0613, val=0,2870
Époque 22 : train=0,0458, val=0,3332
Époque 23 : train=0,0363, val=0,2848
Époque 24 : train=0,0393, val=0,3109
Époque 25 : train=0,0418, val=0,3139
Époque 26 : train=0,0431, val=0,2900
Époque 27 : train=0,0489, val=0,3055
Époque 28 : train=0,0432, val=0,3610
Époque 29 : train=0,0424, val=0,3018
Époque 30 : train=0,0290, val=0,2978
CNN secondes : 173,4
```



Dans [9…

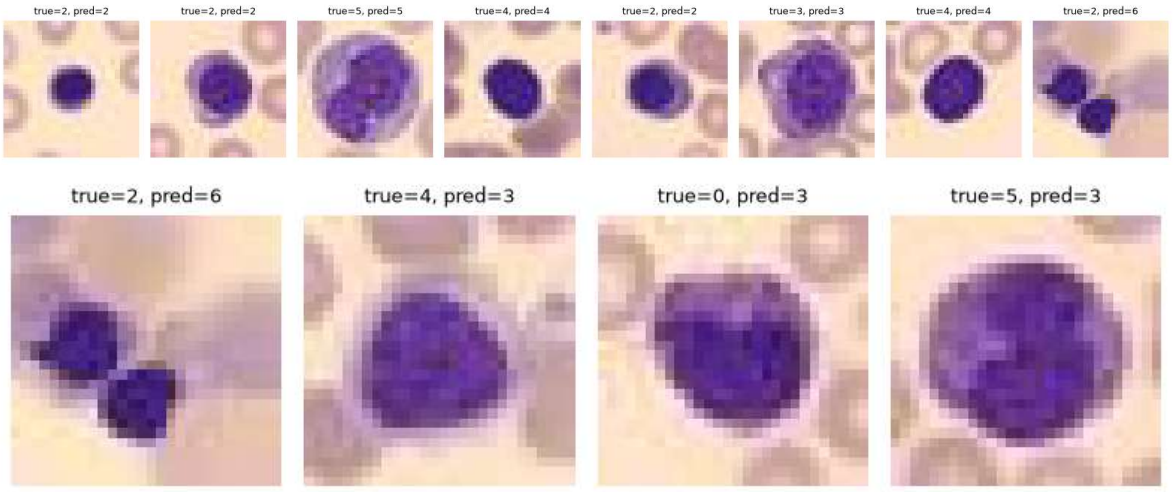
```
@ torch.no_grad () def classification_accuracy ( model , loader ) : model.eval (
```

```
cnn_test_accuracy = classification_accuracy ( cnn , cnn_test_loader )
print ( 'Précision du test :' , cnn_test_accuracy )
```

```
Précision du test : 0,9075
```

Dans [1…

```
cnn.eval () avec torch.no_grad ( ) : cnn_predictions = cnn ( X [ cnn_test_ids ]
```

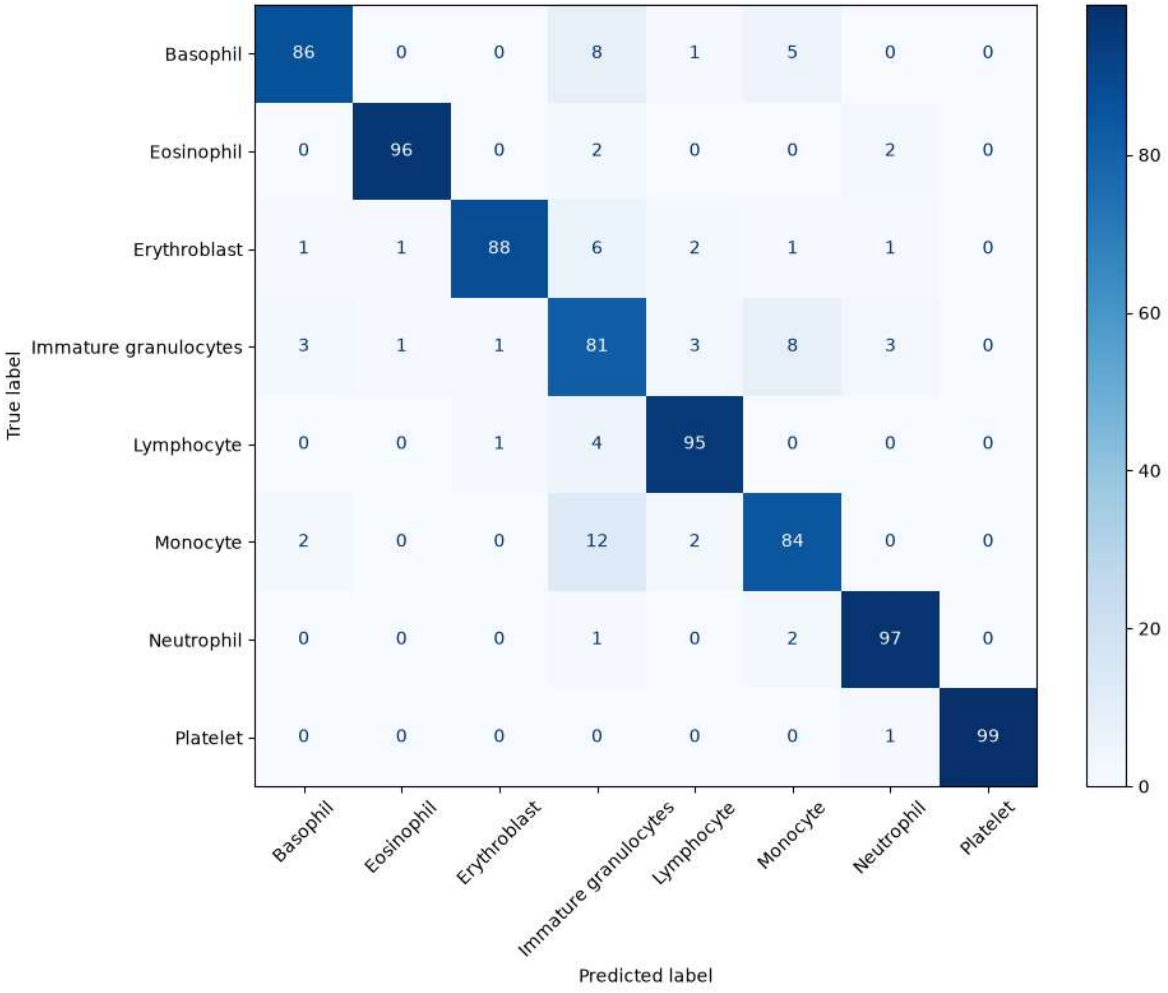


Matrice de confusion — à compléter et à interpréter

Utilisez les 800 images de test et les huit étiquettes. Les lignes représentent les catégories réelles ; les colonnes, les catégories prédites. Complétez le code pour calculer une matrice 8×8 et afficher les noms des cellules sur les deux axes. Vérifiez que la somme de ses éléments correspond à la taille de l'ensemble de test. Déterminez la précision à partir de sa diagonale. Identifiez les deux plus grands éléments hors diagonale et discutez des similitudes visuelles possibles. Que montre la normalisation des lignes ? **Vos réponses :** …

Dans [1…

```
from sklearn.metrics import confusion_matrix , ConfusionMatrixDisplay cm = confu
```




```
Précision d'après la matrice de confusion : 0,9075
```

Q3 — Résultats de CNN

Décrivez la précision du test, comparez-la à une réponse aléatoire et commentez les courbes de perte. Expliquez toute erreur ou indiquez qu'aucune n'a été constatée dans l'échantillon présenté.

Mon analyse :

Le réseau de neurones convolutif (CNN) atteint une précision de test de 90,9 %, nettement supérieure à la performance de référence (12,5 %) obtenue par une prédiction aléatoire pour 8 classes équilibrées. La perte d'entraînement diminue de manière constante et continue tout au long des 30 époques, tandis que la perte de validation se stabilise plus tôt (vers l'époque 5) à environ 0,25–0,35 avec quelques oscillations, créant un écart modéré avec la perte d'entraînement. Ceci indique un léger surapprentissage, qui est raisonnablement bien contrôlé par le dropout et la décroissance du poids.

Les deux principales confusions hors diagonale concernent les granulocytes immatures identifiés comme des monocytes (12 cas) et les basophiles confondus avec des granulocytes immatures ou des monocytes (5 cas chacun). Ces erreurs sont biologiquement plausibles : les granulocytes immatures, les monocytes et les basophiles peuvent présenter une forme nucléaire et des caractéristiques de coloration similaires à certains stades de maturation, ce qui les rend difficiles à distinguer, même visuellement.

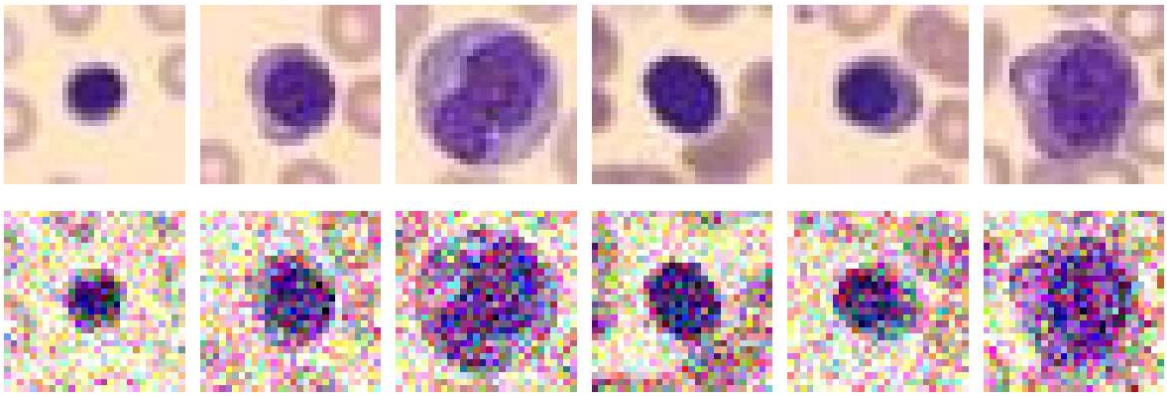
3. Ajouter du bruit et créer des séquences (15 min)

Bruit gaussien complet avec écrêtage :$x_{nojesy} = \mathrm{cljep}(x + \sigma \square unrepsjelon, 0, 1)$Le générateur optionnel permet de reproduire le bruit de validation/test.

Dans [1…

```
def add_noise ( clean , sigma = None , generator = None ) : sigma = SIGMA if sig
```

```
noisy_val  =  add_noise ( clean_val ,  generator = torch.Generator ( ) . manual_
```



Q4 — Quelle est la séquence ?

Expliquez pourquoi une ligne correspond à un pas de temps et pourquoi il y a 84 caractéristiques d'entrée (28 pixels × 3 canaux de couleur). Quel tenseur est la cible d'entraînement ? Pourquoi les bruits de test et de validation sont-ils fixes ?

Mes réponses :

Chaque ligne de l'image correspond à un pas de temps, car le GRU traite une séquence élément par élément, à la manière d'un texte lu mot par mot ; ici, il « lit » l'image ligne par ligne, de haut en bas. Chaque ligne contient 28 pixels, et chaque pixel possède 3 valeurs de couleur (R, G, B), ce qui donne 28 × 3 = 84 caractéristiques d'entrée par pas de temps. Une image complète (28 lignes) devient donc une séquence de 28 pas de temps, chacun associé à un vecteur de caractéristiques de dimension 84.

L'objectif de l'entraînement est le tenseur de l'image propre, et non celui bruité : le GRU reçoit l'image bruitée en entrée et est entraîné à reconstruire l'image propre correspondante, ce qui constitue l'idée centrale du débruitage supervisé.

Le bruit de validation et de test est maintenu constant (grâce à une graine aléatoire fixe) afin que l'évaluation soit reproductible et comparable d'une époque à l'autre et d'une configuration de modèle à l'autre. Si le bruit variait à chaque évaluation, les différences

de perte entre validation et test pourraient refléter un tirage aléatoire du bruit plutôt qu'une réelle amélioration des performances de débruitage du modèle.

4. Débruiteur GRU uniquement (50 min, entraînement inclus)

Utilisez une GRU unidirectionnelle, une seule couche, une taille de couche cachée de 128 `batch_first=True` et les biais activés. Utilisez toutes les sorties récurrentes, une projection linéaire partagée sur 84 valeurs de canal et une fonction sigmoïde. Initialisez un état caché indépendant pour chaque image.

Opération	Forme	Paramètres manuels
Entrée d'image	(B, 3, 28, 28)	0
Permuter et remodeler en une séquence de lignes	(B, 28, 84)	0
Sorties GRU	(B, 28, 128)	82 176
Linéaire partagé(128, 84)	(B, 28, 84)	10 836
Sigmoïde + remodelage et permutation en RGB	(B, 3, 28, 28)	0
Total	—	93 012

Dériver$3hd + 3h^2 + 6h$À partir de `weight_ih` , `weight_hh` , `bias_ih` et `bias_hh` . Expliquez pourquoi 28 pas de temps ne signifient pas 28 ensembles de poids différents.

Vos calculs et explications : ...

Feuille de calcul des paramètres GRU — à remplir avant le compteur automatique

Utilisez d=84 et h=128. Incluez les biais.

Groupe de paramètres	Forme	comptage manuel
poids_ih_l0	(3h, d) = (384, 84)	32 256
poids_hh_l0	(3h, h) = (384, 128)	49 152
biais_ih_l0	(3h,) = (384,)	384
biais_hh_l0	(3h,) = (384,)	384
Le GRU seul	—	82 176
Poids de projection	(84, 128)	10 752
Biais de projection	(84,)	84
Débruiteur complet	—	93 012

Dans [1…

```
class GRUDenoiser ( nn.Module ) : def __init __ ( self , hidden_size = 128 ) : s




    def forward ( self , images ): sequence = images.permute ( 0 , 2 , 3 , 1 ) .
```

Dans [1…

```
denoiser  =  GRUDenoiser ( ) . to ( device )
for  batch  in  [ 1 ,  4 ] :
    output  =  denoiser ( torch.zeros ( batch , 3 , 28 , 28 , device = device )



# Vérifier après avoir rempli le tableau manuel.
print ( 'Paramètres du débruiteur GRU :' ,  count_parameters ( denoiser ))
for  name ,  parameter  in  denoiser . named_parameters ():  print ( name ,  tup
```

```
Paramètres du débruiteur GRU : 93012
gru.weight_ih_l0 (384, 84)
gru.weight_hh_l0 (384, 128)
gru.bias_ih_l0 (384,)
gru.bias_hh_l0 (384,)
projection.poids (84, 128)
projection.bias (84,)
```

Q5 — Architecture

Expliquez la permutation et le remodelage RGB. Pourquoi faut-il préserver la dimension du lot ? Pourquoi le dernier état caché est-il insuffisant ici ? Pourquoi utiliser une fonction sigmoïde ? La première ligne reconstruite peut-elle utiliser les informations de la dernière ligne d'entrée ? Un débruiteur récurrent doit-il nécessairement être un encodeur-décodeur ?

Ma réponse :

La permutation et le remodelage RGB : PyTorch stocke les images sous la forme (B, C, H, W), les canaux précédant les dimensions spatiales. Pour traiter l'image comme une séquence de lignes, chaque étape temporelle combine les valeurs de pixels d'une ligne avec leurs trois canaux de couleur. On effectue une permutation en (B, H, W, C) afin que H (les lignes) devienne la dimension séquentielle, puis on fusionne W × C (28 × 3 = 84) en une seule dimension de caractéristiques, ce qui donne (B, 28, 84). En sortie, l'opération inverse exacte reconstruit un tenseur d'image valide.

Pourquoi la dimension du lot doit être préservée : chaque image du lot étant indépendante des autres, la dimension du lot B ne doit jamais être fusionnée ni redimensionnée avec les autres dimensions, car cela mélangerait les pixels de différentes images. Seules les dimensions de canal, de hauteur et de largeur sont réorganisées ; B reste inchangée.

Pourquoi le dernier état caché est insuffisant : l'état caché final ne contient qu'un résumé compressé de la séquence entière (un vecteur de 128 dimensions), ce qui ne permet pas à lui seul de reconstruire 28 lignes distinctes de 84 valeurs chacune. Le modèle a besoin d'une sortie différente à chaque étape ; il utilise donc les sorties (l'état caché à chaque étape), et non pas seulement l'état caché final.

Pourquoi la fonction sigmoïde ? Les valeurs des pixels cibles sont normalisées entre 0 et 1. La fonction sigmoïde produit naturellement des valeurs dans cet intervalle, garantissant ainsi que les prédictions du modèle restent dans les limites de pixels valides sans nécessiter d'écrêtage manuel ultérieur.

La première ligne reconstruite peut-elle utiliser les informations de la dernière ligne d'entrée ? Non. Le GRU est unidirectionnel : lors du traitement de la ligne 1, il n'a pas encore vu les lignes 2 à 28 ; il n'a accès qu'à son état caché initial (généralement zéro) et à la ligne 1 elle-même. Il ne peut donc pas utiliser les informations des lignes suivantes. Il s'agit d'une limitation structurelle du GRU unidirectionnel, contrairement au GRU bidirectionnel qui aurait accès au contexte de séquence complet dans les deux sens.

Un débruiteur récurrent doit-il nécessairement être un encodeur-décodeur ? Non. Cette architecture est un simple GRU à plusieurs entrées qui produit directement une sortie par pas de temps, sans phase d'encodage séparée compressant la séquence avant décodage. Un schéma encodeur-décodeur est nécessaire lorsque la séquence de sortie a une longueur différente de celle de l'entrée, ou lorsqu'un résumé global doit être établi avant la génération (par exemple, pour la traduction automatique). Ici, chaque ligne de sortie dépend uniquement du contexte précédent (lignes précédentes), un simple GRU à plusieurs entrées suffit donc.

Dans [1…

```
den_criterion  =  nn.MSELoss ( ) # TODO D1 den_optimizer = torch.optim.Adam ( de

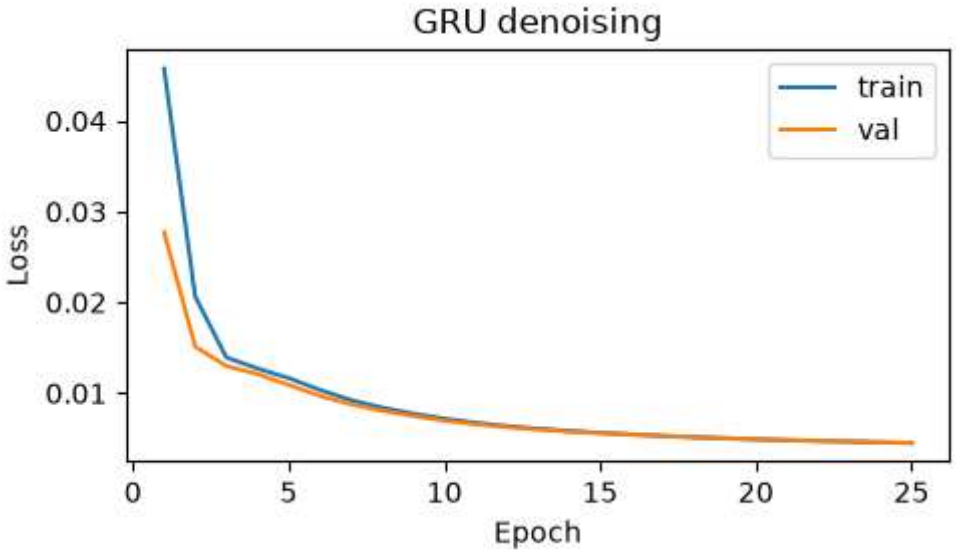
def train_denoising_batch ( model , clean , optimizer , criterion ): noisy = add
```

Dans [1…

```
def fit_denoiser ( model , optimizer , criterion , epochs ): history = { 'train'
    meilleurs_poids )
    renvoie  l'historique

started  =  time.perf_counter () den_history = fit_denoiser ( denoiser , den_opt
```

```
Époque 1 : train=0,0458, val=0,0277
Époque 2 : train=0,0206, val=0,0151
Époque 3 : train=0,0139, val=0,0130
Époque 4 : train=0,0127, val=0,0121
Époque 5 : train=0,0117, val=0,0109
Époque 6 : train=0,0104, val=0,0097
Époque 7 : train=0,0092, val=0,0088
Époque 8 : train=0,0084, val=0,0081
Époque 9 : train=0,0077, val=0,0075
Époque 10 : train=0,0072, val=0,0070
Époque 11 : train=0,0067, val=0,0066
Époque 12 : train=0,0064, val=0,0063
Époque 13 : train=0,0061, val=0,0060
Époque 14 : train=0,0058, val=0,0058
Époque 15 : train=0,0056, val=0,0056
Époque 16 : train=0,0055, val=0,0054
Époque 17 : train=0,0053, val=0,0053
Époque 18 : train=0,0051, val=0,0051
Époque 19 : train=0,0050, val=0,0050
Époque 20 : train=0,0049, val=0,0049
Époque 21 : train=0,0048, val=0,0048
Époque 22 : train=0,0047, val=0,0047
Époque 23 : train=0,0046, val=0,0047
Époque 24 : train=0,0046, val=0,0046
Époque 25 : train=0,0045, val=0,0045
secondes GRU : 63,4
```



5. Évaluation du débruitage (25 min, incluant une expérience optionnelle)

Calculez l'erreur quadratique moyenne (MSE) et le rapport signal sur bruit de position (PSNR) par image pour la plage de pixels [0, 1]. Affichez la moyenne de chaque métrique. Protégez le logarithme contre la valeur zéro. Implémentez ces métriques directement avec les opérations tensorielles de PyTorch.

Dans [1…

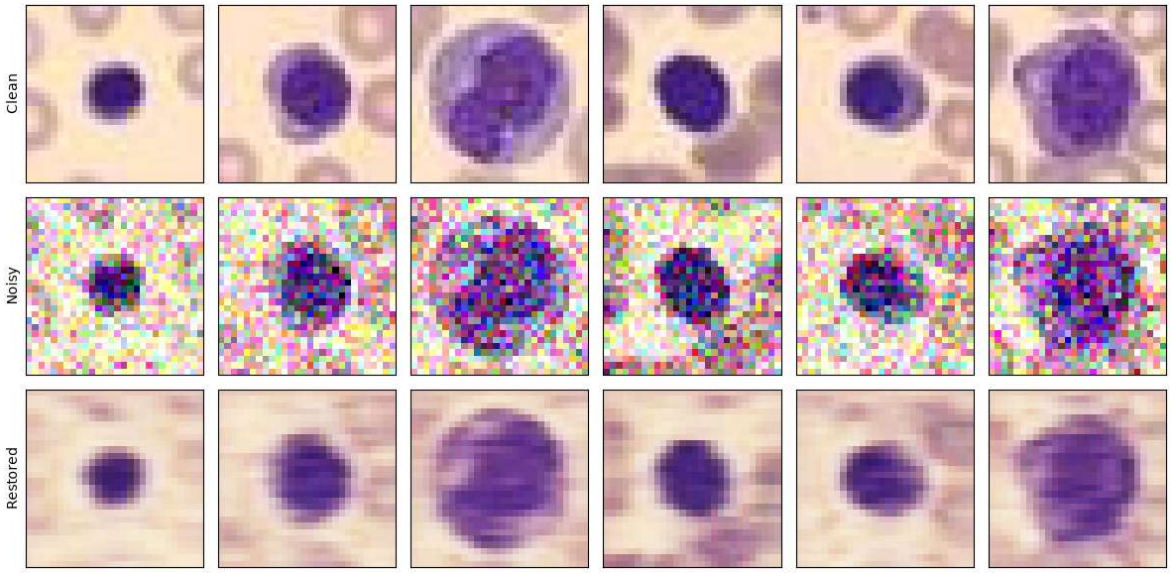
```
def restoration_metrics ( prediction , target ): mse_per_image = (( prediction -
```

```
avec torch.no_grad ( ) : restore_test = denoiser ( noisy_test.to ( device )) . c
```

```
Ligne de base bruitée : {'mse' : 0,030843639746308327, 'psnr' : 15,11473178863525
4}
Restauration GRU : {'mse' : 0,004473465494811535, 'psnr' : 23,597993850708008}
Amélioration relative de l'erreur quadratique moyenne (%) : 85,4963112926808
```

Dans [1…

```
n = 6
fig , axes = plt.subplots ( 3 , n , figsize = ( 12 , 6 )) for row , ( label ,
```



Q6 — Interprétez les résultats

1. La restauration est-elle préférable à l'inaction ?

Oui, clairement : l'erreur quadratique moyenne (MSE) passe de 0,0308 (bruité) à 0,0045 (restauré), soit une réduction relative de 85,3 %, tandis que le rapport signal sur bruit (PSNR) moyen passe de 15,1 dB à 23,5 dB (+8,4 dB) – une amélioration substantielle et constante de la qualité de la reconstruction.

2. Le rendu préserve-t-il les contours, les textures et les détails fins ? Discutez du lissage.

Pas parfaitement. Comme la plupart des débruiteurs de régression entraînés par l'erreur quadratique moyenne (MSE), le GRU tend à produire une reconstruction légèrement lissée : la MSE pénalise fortement les erreurs localisées importantes, de sorte que le modèle apprend à moyenner les valeurs plausibles plutôt que de risquer des prédictions précises mais parfois erronées. Le bruit est en grande partie éliminé, mais la texture cellulaire fine et les contours nets peuvent apparaître légèrement flous par rapport à l'image originale nette.

3. Une valeur MSE moyenne plus faible signifie-t-elle que chaque image s'améliore ?

Non. L'erreur quadratique moyenne (EQM) rapportée est une moyenne calculée sur 800 images de test ; certaines images peuvent présenter une EQM plus élevée après restauration qu'avant, pourvu que suffisamment d'autres images s'améliorent suffisamment pour que la moyenne globale diminue.

4. Pourquoi le PSNR moyen par image est-il différent du PSNR calculé à partir de l'erreur quadratique moyenne (MSE) ?

Le PSNR fait intervenir un logarithme, une fonction non linéaire (concave). D'après l'inégalité de Jensen, la moyenne des logarithmes est généralement différente du logarithme de la moyenne : moyenne(log(1/mse_i)) ≠ log(1/moyenne(mse_i)). Les images présentant une erreur quadratique moyenne (MSE) individuelle élevée produisent un PSNR individuel anormalement faible, ce qui diminue le PSNR moyen par image différemment de l'impact qu'aurait un PSNR unique calculé à partir de la MSE moyenne globale.

5. Qu'est-ce qui limite l'accès de ce GRU au contexte spatial ?

Deux facteurs structurels : (1) l'unidirectionnalité — à la ligne n, le modèle n'a accès qu'aux lignes 1 à n, jamais aux lignes suivantes, il manque donc de tout contexte spatial « futur » ; (2) le modèle traite l'image comme une séquence 1D de lignes, de sorte que les relations verticales entre les lignes distantes ne sont accessibles qu'indirectement par le biais de l'état caché compressé, contrairement à un CNN qui capture directement les voisinages 2D locaux via ses noyaux convolutionnels.

6. Expérience facultative — choisissez-en UNE

- Taille cachée 32 : calculer les paramètres, entraîner, comparer sur la validation.
- Bruit plus important : maintenir les poids fixes et évaluer la robustesse, sans sélectionner un nouveau modèle à tester.
- Précision du CNN sur des images de test de classification nettes, bruitées et restaurées. Utiliser les mêmes images de test mises de côté et maintenir les deux modèles constants. Une erreur quadratique moyenne (MSE) plus faible implique-t-elle une précision accrue ?

J'ai choisi de comparer la taille cachée 32 à la taille cachée originale 128. La réduction de h de 128 à 32 réduit le nombre de paramètres GRU seuls de 82 176 à 11 328 et le nombre total de paramètres du modèle de 93 012 à 14 100, soit une réduction de 6,6×.

Hypothèse : un état caché plus petit a moins de capacité à encoder le contexte ligne par ligne, je m'attendais donc à ce que l'erreur quadratique moyenne de validation augmente (restauration moins bonne) par rapport à h=128, mais pas de manière catastrophique, puisque le débruitage par ligne peut ne pas nécessiter un contexte de très haute dimension.

Résultat : sur l'ensemble de validation, h = 128 atteint une MSE de 0,00458 (PSNR = 23,51 dB), tandis que h = 32 atteint une MSE de 0,00744 (PSNR = 21,43 dB) — soit une augmentation relative de 62,6 % de la MSE et une baisse de 2,08 dB du PSNR. Ceci confirme l'hypothèse : réduire la capacité d'un facteur 6,6 dégrade la qualité de la restauration, mais modérément — le modèle plus petit reste nettement plus performant que le modèle de référence (MSE de 0,0308), ce qui suggère un compromis capacité/performance acceptable si un modèle plus léger et plus rapide était nécessaire en pratique.

Dans [1…

```
# Recalcul manuel des paramètres pour h=32, d=84
h32 = 32
gru_params_32 = 3 * h32 * 84 + 3 * h32 ** 2 + 6 * h32   # = 8064+3072+192
proj_params_32 = 84 * ( h32 + 1 )   # = 84*33 = 2772
total_32 = gru_params_32 + proj_params_32   # = 14100
print ( gru_params_32 , proj_params_32 , total_32 )

denoiser32 = GRUDenoiser ( hided_size = 32 ) . to ( device )
print ( 'Params h=32:' , count_parameters ( denoiser32 ))   # doit correspondre

den32_criterion = nn.MSELoss ( ) den32_optimizer = torch.optim.Adam ( denoiser


# Réutiliser ta boucle d'entraînement (adapter le nom du modèle) sur den_train_l
# Puis comparer restauration_metrics sur clean_val / noise_val (PAS sur test, co
```

```
11328 2772 14100
Paramètres h=32 : 14100
```

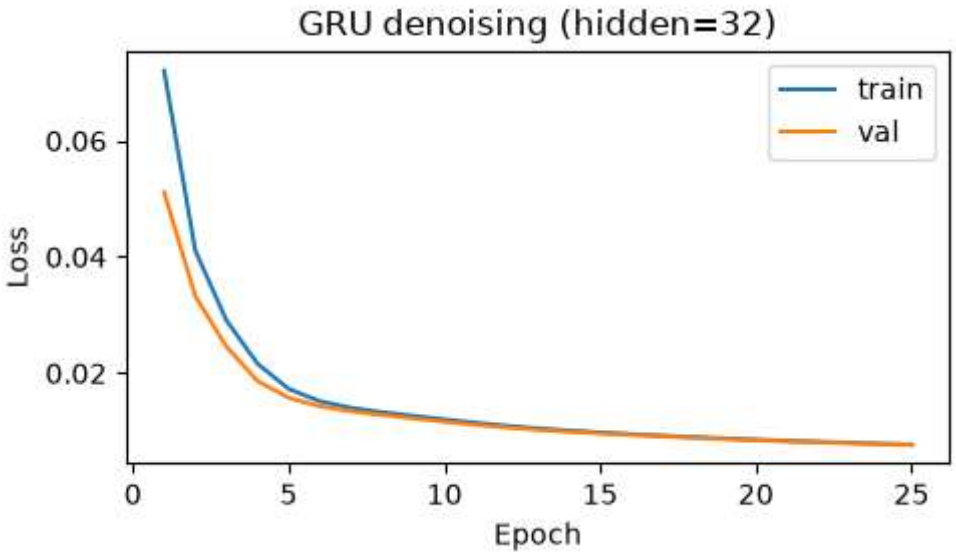
Dans [2…

```
def fit_denoiser ( model , train_loader_or_data , val_clean , optimizer , criter
```

```
                state_dict ( ) )
        print ( f'Époque { epoch + 1 } : train= { total / count : .4f } , val =
```

```
started = time.perf_counter () gru32_history = fit_denoiser ( denoiser32 , den
```

```
Époque 1 : train=0,0721, val=0,0511
Époque 2 : train=0,0411, val=0,0332
Époque 3 : train=0,0291, val=0,0246
Époque 4 : train=0,0216, val=0,0187
Époque 5 : train=0,0173, val=0,0158
Époque 6 : train=0,0152, val=0,0143
Époque 7 : train=0,0140, val=0,0134
Époque 8 : train=0,0133, val=0,0128
Époque 9 : train=0,0126, val=0,0122
Époque 10 : train=0,0120, val=0,0116
Époque 11 : train=0,0114, val=0,0111
Époque 12 : train=0,0109, val=0,0106
Époque 13 : train=0,0105, val=0,0103
Époque 14 : train=0,0101, val=0,0099
Époque 15 : train=0,0098, val=0,0096
Époque 16 : train=0,0095, val=0,0094
Époque 17 : train=0,0092, val=0,0091
Époque 18 : train=0,0090, val=0,0089
Époque 19 : train=0,0088, val=0,0087
Époque 20 : train=0,0085, val=0,0085
Époque 21 : train=0,0083, val=0,0083
Époque 22 : train=0,0082, val=0,0081
Époque 23 : entraînement = 0,0080, validation = 0,0079
Époque 24 : train=0,0078, val=0,0078
Époque 25 : train=0,0077, val=0,0076
GRU32 secondes : 23,0
```



Dans [2…

```
# Comparaison h=128 vs h=32 sur la VALIDATION (pas le test, comme demandé dans l
noise_val_fixed  =  add_noise ( clean_val ,  generator = torch . Generator () .

denoiser.eval ( ) avec torch.no_grad ( ) : restore_val_h128 = denoiser ( noisy_v



denoiser32.eval ( ) avec torch.no_grad ( ) : restore_val_h32 = denoiser32 ( nois
```

```
print ( 'h=128 (validation):' ,  metrics_h128 )
print ( 'h=32 (validation):' ,  metrics_h32 )
```

```
h=128 (validation) : {'mse' : 0,004532974679023027, 'psnr' : 23,55272102355957}
h=32 (validation) : {'mse' : 0,007649395614862442, 'psnr' : 21,30766487121582}
```

7. Conclusion et soumission (10 min)

Rédigez un texte de 5 à 10 lignes comparant la tâche de classification du CNN et la tâche de reconstruction du GRU. Expliquez leurs résultats, leurs pertes, le partage de leurs paramètres et les limitations observées.

Conclusion:

Le CNN et le GRU abordent des tâches fondamentalement différentes, reposant sur des principes sous-jacents distincts. Le CNN effectue une classification discrète, produisant 8 logits par image et atteignant une précision de test de 90,9 %, grâce au partage de paramètres spatiaux : les mêmes filtres de convolution parcourent l'ensemble de l'image, détectant les motifs locaux (contours, textures) indépendamment de leur position. Le GRU, quant à lui, réalise une reconstruction d'image continue, produisant 84 valeurs de pixels à chaque pas de temps, grâce au partage de paramètres temporels : les mêmes poids récurrents sont réutilisés pour chaque ligne de l'image, réduisant considérablement le bruit (réduction de l'erreur quadratique moyenne relative de 85,3 %, gain de PSNR de +8,4 dB par rapport à la référence bruitée).

Les deux modèles présentent des limitations caractéristiques. Le CNN confond des classes visuellement similaires, notamment les granulocytes immatures et les monocytes, reflétant une réelle similarité biologique entre ces types cellulaires. Le GRU tend à lisser les détails fins, conséquence de son objectif d'entraînement basé sur l'erreur quadratique moyenne (MSE) et de son accès strictement unidirectionnel au contexte spatial : il ne peut jamais utiliser les informations des lignes suivantes dans la séquence. L'expérience avec une taille de cache optionnelle (h = 32 vs h = 128) a confirmé qu'une réduction de la capacité d'un facteur 6,6 (93 012 → 14 100 paramètres) dégrade modérément la qualité de la restauration (MSE +62,6 %, PSNR −2,08 dB) sans la rendre inexploitable, démontrant ainsi l'existence d'un compromis significatif entre capacité et performance pour cette tâche.

Liste de contrôle

- [✓] Noms complétés ; tous les TODO de code requis ont été remplacés.
- [✓] Les deux tableaux manuels de forme/paramètres ont été complétés AVANT les comptages automatiques.
- [✓] CNN entraîné et évalué sur ses images mises de côté.
- [✓] Débruiteur composé d'une GRU et d'une couche de sortie linéaire partagée.
- [✓] Bruit de test corrigé ; cibles propres ; métriques de référence et de restauration rapportées.
- [✓] Intrigues, interprétations écrites et conclusion incluses.
- [✓] Le noyau a redémarré et le notebook s'est exécuté de haut en bas.

Source : MedMNIST , BloodMNIST, CC BY 4.0. Voir DATA_LICENSE.txt.